

endian, Just endian

Contents

- [Introduction](#)
- [Proposal](#)
- [Implementation](#)
- [Wording](#)

Changes from R0

- Tweak wording per LWG guidance.

Introduction

A ancient, time-honored tradition among C++ programmers is detecting the endian of their execution environment:

[Detecting endianness programmatically in a C++ program](#)

There are many hacks that work *most* of the time. None of them seem bullet proof. Few give the answer as a compile-time constant. And in every single case:

The compiler knows the answer!

It is a no-brainer for the committee to provide an API so that the programmer can query the implementation for an answer to this common question.

Proposal

Put into `<type_traits>`:

```
enum class endian
{
    little = __ORDER_LITTLE_ENDIAN__,
    big    = __ORDER_BIG_ENDIAN__,
    native = __BYTE_ORDER__
};
```

with appropriate constants for platforms that don't define `__BYTE_ORDER__` et al. That's it. That's the entire proposal.

Objection #1: Some modern processors can switch endian at run time. How can this be a compile-time constant?

No operating system tolerates switching endian at run time once an application has launched. Every compiler knows what endian it has to target.

Objection #2: Where are the functions to translate scalars between native endian and big or little endian?

Those are good features to have. But they do not represent everything that is needed. And they can more easily be built on top of `class endian` once it exists. At a minimum we need `class endian` as soon as possible (1998 would be nice). This proposal aims to get the minimum required functionality in as quickly as possible.

Objection #3: I am quite sure that PDP endian is on the way back in. PDP is neither big nor little endian. This proposal doesn't handle that!

1. This proposal handles "mixed endian" today by ensuring that `endian::native` equals neither `endian::big` nor `endian::little`.
2. Today, no C++14 compiler targets a machine that is not big endian or little endian.
3. If tomorrow we need another flavor of endian, a new member could easily be added to `class endian` with no

backwards compatibility problems (e.g. `endian::pdp`).

How do I use this?

You can either build compile-time traits with this, or use it at run time. For example:

```
if (endian::native == endian::big)
    // handle big endian
else if (endian::native == endian::little)
    // handle big endian
else
    // handle mixed endian
```

Why not just a macro?

This API was chosen because there are a few times when a program needs to do *more* than just query endianness. Sometimes a program needs to either *declare* or *request* endianness. This API provides a universal way of doing so. For example a fingerprinting hash object such as [SHA-2](#) traffics in big endian. This might be advertised like:

```
class sha256
{
public:
    static constexpr std::endian endian = std::endian::big;
    // ...
```

The type `sha256` may need to meet a type concept that requires a public nested value of type `std::endian` that specifies how input should be preprocessed, or that specifies how some of the output is encoded. Details like this are crucial for inter-machine communication.

Implementation

```
enum class endian
{
#ifdef _WIN32
    little = 0,
    big    = 1,
    native = little
#else
    little = __ORDER_LITTLE_ENDIAN__,
    big    = __ORDER_BIG_ENDIAN__
    native = __BYTE_ORDER__,
#endif
};
```

On the platform on which I'm writing this, this preprocesses to:

```
enum class endian
{
    little = 1234,
    big    = 4321,
    native = 1234
};
```

Wording

1. Add to [meta.type.synop].

```
enum class endian
{
    little = see below,
    big    = see below,
    native = see below
};
```

2. Add a new section after [meta.logical]:

23.15.9 Endian [meta.endian]

Two common methods of byte ordering in multibyte scalar types are big-endian and little-endian in the execution environment. Big-endian is a format for storage of binary data in which the most significant byte is placed first, with the rest in descending order. Little-endian is a format for storage of binary data in which the least significant byte is placed first, with the rest in ascending order. This subclause describes the endianness of the scalar types of the execution environment.

```
enum class endian
{
    little = see below,
    big    = see below,
    native = see below
};
```

If all scalar types have size 1, then all of `endian::little`, `endian::big`, and `endian::native` have the same value. Otherwise `endian::little` shall not be equal to `endian::big`. If all scalar types are big-endian, `endian::native` shall be equal to `endian::big`. If all scalar types are little-endian, `endian::native` shall be equal to `endian::little`. Otherwise `endian::native` shall have a value that is not equal to either of `endian::big` nor `endian::little`.